

Phân loại rủi ro khách hàng mua bảo hiểm

A42718 - Lê Thảo Quyên

A43457 - Đỗ Thị Phương



Giới thiệu về Prudential

Prudential là một trong những nhà phát hành bảo hiểm nhân thọ lớn nhất tại Mỹ, giúp mọi người ở mọi lứa tuổi và nền tảng phát triển và bảo vệ tài sản của họ thông qua nhiều sản phẩm và dịch vụ, bao gồm bảo hiểm nhân thọ, niên kim, dịch vụ liên quan đến hưu trí, quỹ tương hỗ và quản lý đầu tư.

Thách thức: Trong một thế giới mua sắm chỉ với một cú nhấp chuột, quy trình nộp đơn bảo hiểm nhân thọ đã trở nên lạc hậu. Khách hàng cung cấp thông tin chi tiết để xác định phân loại rủi ro và đủ điều kiện, bao gồm cả việc lên lịch các cuộc kiểm tra y tế, một quy trình mất trung bình 30 ngày.

Kết quả? Người tiêu dùng cảm thấy không hào hứng. Prudential muốn làm cho quy trình nhận báo giá nhanh hơn và ít tốn công sức hơn.

Business Case

Business Case xác định vấn đề kinh doanh, các bên liên quan, phạm vi, giả định, phụ thuộc, ảnh hưởng và yếu tố thành công.

Vấn đề/Mục tiêu kinh doanh

- Phân loại rủi ro KH mua bảo hiểm
- Tối ưu hóa quy trình thực hiện
- Cải thiện khả năng ra quyết định

Các bên liên quan

- Công ty, đại lý bảo hiểm
- Cơ quan quản lý
- Nhà phân tích dữ liệu

Phạm vi

Bộ dữ liệu gồm 59 381 quan sát.

Business Case

Business Case xác định vấn đề kinh doanh, các bên liên quan, phạm vi, giả định, phụ thuộc, ảnh hưởng và yếu tố thành công.

Giả định và phụ thuộc

Giả định:

- Dữ liệu chính xác
- Chi phí như nhau
- Thời gian như nhau

Phụ thuộc: Khả năng phát triển phần mềm

Ảnh hưởng

- Tối ưu hóa quy trình thực hiện
- Cải thiện khả năng ra quyết định

Yếu tố thành công

- AUC cao (>70%)
- Dễ triển khai
- Dễ giải thích

Mục tiêu kỹ thuật



Mô hình dự đoán
tốt những KH
có rủi ro
nghiêm trọng



Phân loại rủi ro
của KH mua
bảo hiểm theo
mức độ từ 1 → 8



AUC cao > 70%



Tỷ lệ phân chia dữ liệu

Phân chia dữ liệu thành các tập Train, Validation và Test để đánh giá hiệu quả của mô hình.

Tập Train	Tập Validation	Tập Test
60%	20%	20%



Chuẩn bị dữ liệu

Chuẩn bị dữ liệu bao gồm tổng quan dữ liệu, xử lý dữ liệu thiếu, feature engineering và feature selection.

1 Tổng quan dữ liệu

Phân tích dữ liệu về số lượng quan sát, biến, và tỷ lệ rủi ro bảo hiểm ở mức cao nhất.

2 Xử lý dữ liệu thiếu

Xử lý các giá trị thiếu trong dữ liệu, bằng cách xóa bỏ hoặc thay thế bằng giá trị phù hợp.

3 Feature Engineering

Tạo thêm các cột mới cho dữ liệu, bằng cách kết hợp hoặc biến đổi các cột hiện có.

4 Feature Selection

Loại bỏ các biến không cần thiết, bằng cách sử dụng kỹ thuật lựa chọn biến phù hợp.

Tổng quan dữ liệu

- 59 381 quan sát
- 127 biến đầu vào
- 1 biến mục tiêu **Response**, đây là thước đo thứ bậc về rủi ro gồm 8 mức độ
- 32.82% **Response** ở mức 8 - mức cao nhất



Proportion of zeroes in Employment_Info_1 is: 0.04 %.
Proportion of zeroes in Employment_Info_4 is: 11.23 %.
Proportion of zeroes in Employment_Info_6 is: 18.29 %.
Proportion of zeroes in Insurance_History_5 is: 42.65 %.
Proportion of zeroes in Family_Hist_2 is: 48.11 %.
Proportion of zeroes in Family_Hist_3 is: 57.79 %.
Proportion of zeroes in Family_Hist_4 is: 32.16 %.
Proportion of zeroes in Family_Hist_5 is: 70.47 %.
Proportion of zeroes in Medical_History_1 is: 15.07 %.
Proportion of zeroes in Medical_History_10 is: 99.08 %.
Proportion of zeroes in Medical_History_15 is: 75.06 %.
Proportion of zeroes in Medical_History_24 is: 93.5 %.
Proportion of zeroes in Medical_History_32 is: 98.18 %.

Xử lý dữ liệu thiếu

- Xóa bỏ các cột có tỷ lệ giá trị bị thiếu trong tập train lớn hơn 40%.
- Thực hiện ước lượng lặp trên các cột có giá trị bị thiếu còn lại.

Feature Engineering

Sử dụng các kỹ thuật Mã hóa One-Hot, Chuẩn hóa và Phân cụm K-Means để tạo ra các đặc trưng mới trong tập dữ liệu

Mã hóa One-Hot

Thực hiện mã hóa one-hot cho cột Product_Info_2, sao cho bất kỳ đầu vào phân loại nào được chuyển đổi thành các giá trị số có thể được hiểu bởi máy, có thể cung cấp cho mô hình phân loại.

```
from sklearn.preprocessing import OneHotEncoder

# Khởi tạo một bộ mã hóa one-hot cho các cột chứa dữ liệu phân loại.
OH_encoder = OneHotEncoder(handle_unknown='ignore', sparse=False)
# -> đặt handle_unknown='ignore' -> tránh lỗi khi dữ liệu xác thực chứa các lớp không có trong dữ liệu huấn luyện,
# -> đặt sparse=False đảm bảo rằng các cột đã mã hóa được trả về dưới dạng mảng numpy (thay vì ma trận thưa).
# Xác định cột cần mã hóa
OH_col = ['Product_Info_2']

# Sử dụng bộ mã hóa one-hot để biến đổi các cột dữ liệu phân loại (categorical data).
OH_col_train = pd.DataFrame(OH_encoder.fit_transform(copy_X_train_imputed[OH_col]))
OH_col_valid = pd.DataFrame(OH_encoder.transform(copy_X_valid_imputed[OH_col]))
OH_col_test = pd.DataFrame(OH_encoder.transform(copy_X_test_imputed[OH_col]))
# fit_transform được sử dụng trên tập huấn luyện để vừa mã hóa vừa học các lớp.
# transform được sử dụng trên các tập xác thực và kiểm tra để mã hóa mà không thay đổi lớp đã học từ tập huấn luyện.

# Mã hóa one-hot loại bỏ chỉ mục; gán lại chỉ mục gốc.
# -> giữ nguyên cấu trúc dữ liệu.
OH_col_train.index = copy_X_train_imputed.index
OH_col_valid.index = copy_X_valid_imputed.index
OH_col_test.index = copy_X_test_imputed.index

# Thêm nhãn cột trở lại, sử dụng hàm get_feature_names().
# -> dễ dàng nhận diện các giá trị.
OH_col_train.columns = OH_encoder.get_feature_names(OH_col)
OH_col_valid.columns = OH_encoder.get_feature_names(OH_col)
OH_col_test.columns = OH_encoder.get_feature_names(OH_col)

# Tạo các dataframe chỉ bao gồm các đặc trưng/cột số
# (các cột này sẽ được kết hợp với các dataframe đã mã hóa one-hot).
# hay Loại bỏ cột Product_Info_2 từ các dataframe gốc để chỉ giữ lại các cột số.
copy_X_train_imputed_no_OH_col = copy_X_train_imputed.drop(OH_col, axis=1)
copy_X_valid_imputed_no_OH_col = copy_X_valid_imputed.drop(OH_col, axis=1)
copy_X_test_imputed_no_OH_col = copy_X_test_imputed.drop(OH_col, axis=1)

# Kết hợp các cột đã mã hóa one-hot với các đặc trưng/cột số hiện có.
# -> tạo ra các tập dữ liệu hoàn chỉnh cho mô hình học máy.
X_train_enc = pd.concat([copy_X_train_imputed_no_OH_col, OH_col_train], axis=1)
X_valid_enc = pd.concat([copy_X_valid_imputed_no_OH_col, OH_col_valid], axis=1)
X_test_enc = pd.concat([copy_X_test_imputed_no_OH_col, OH_col_test], axis=1)
```

Chuẩn hóa

```
from sklearn.preprocessing import MinMaxScaler

# Khởi tạo mô hình MinMaxScaler, sau đó fit với tập dữ liệu huấn luyện (đã mã hóa).
# mô hình này sẽ biến đổi các giá trị trong mỗi cột để nằm trong khoảng từ 0 đến 1.
MM_scaler = MinMaxScaler()
MM_scaler.fit(X_train_enc)

# Sau đó, chuẩn hóa/biến đổi các tập dữ liệu huấn luyện, xác thực và kiểm tra.
X_train_scale = pd.DataFrame(MM_scaler.transform(X_train_enc),
                              index=X_train_enc.index,
                              columns=X_train_enc.columns)

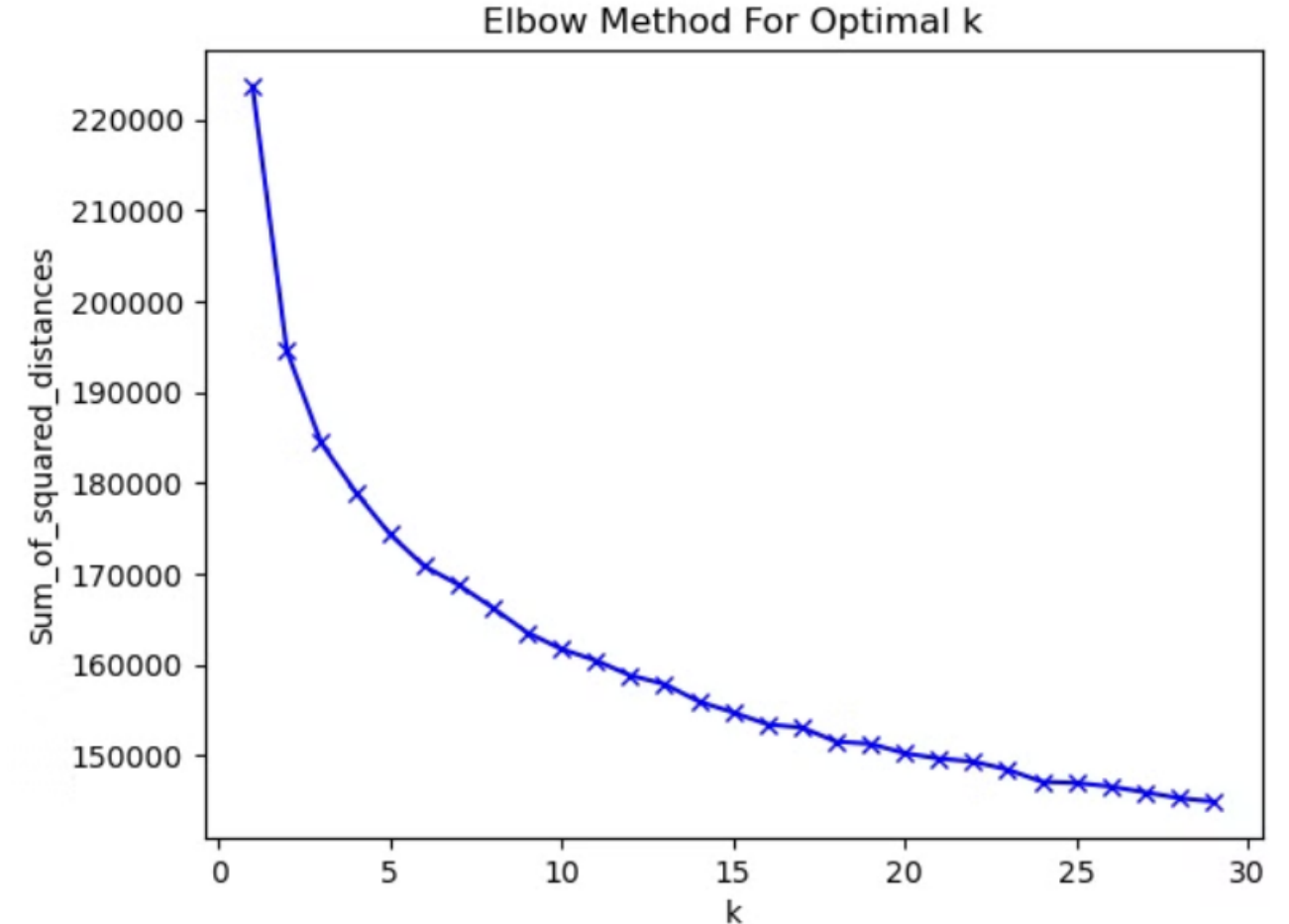
X_valid_scale = pd.DataFrame(MM_scaler.transform(X_valid_enc),
                              index=X_valid_enc.index,
                              columns=X_valid_enc.columns)

X_test_scale = pd.DataFrame(MM_scaler.transform(X_test_enc),
                              index=X_test_enc.index,
                              columns=X_test_enc.columns)
```

Thực hiện chuẩn hóa min-max trên các tập dữ liệu đã mã hóa, sao cho tất cả các đặc trưng nằm trong khoảng từ 0 đến 1. Điều này giúp khi huấn luyện thì các đặc trưng sẽ có phương sai cùng một bậc độ lớn với nhau. Nhờ đó, không có đặc trưng nào chiếm ưu thế trong hàm mục tiêu và ngăn cản mô hình học hỏi từ các đặc trưng khác một cách chính xác như mong đợi.

Phân cụm K-Means

- Sử dụng phân cụm K-means để nhóm các ứng viên dựa trên những điểm chung của họ bằng cách sử dụng tập con huấn luyện, trước khi dự đoán các nhóm cụm trên cả ba tập con.
- Các nhãn cụm này sẽ được đưa vào như một đặc trưng bổ sung trong các tập và có thể thực sự hữu ích trong việc giúp hiểu các phân loại rủi ro của các ứng viên sau này.
- Sử dụng phương pháp phân tích "khủy tay" (elbow-analysis) để xác định số lượng cụm tối ưu cần sử dụng, tránh việc chọn quá nhiều hoặc quá ít, từ đó cải thiện hiệu suất.



- "Khủy tay" của đường cong bắt đầu hình thành tại $k = 15$, nghĩa là việc tăng k lớn hơn giá trị này không mang lại lợi ích đáng kể trong việc giảm độ sai lệch của tập train.

Feature Selection

```
# Nhập các mô-đun chính để thực hiện chọn lọc đặc trưng dựa trên LASSO.
from sklearn.svm import LinearSVC
from sklearn.feature_selection import SelectFromModel

# Thiết lập mô hình Điều chỉnh Lasso (L1) sẽ thực hiện chọn lọc đặc trưng.
linearsvc = LinearSVC(penalty="l1", dual=False, tol=1e-3, C=1e-2, random_state=0).fit(X_valid_KMeans, y_valid)
model = SelectFromModel(linearsvc, prefit=True)

# Giảm tập dữ liệu xuống các đặc trưng quan trọng nhất, sử dụng mô hình điều chỉnh ở trên.
X_valid_L1 = model.transform(X_valid_KMeans)

# Chuyển đổi tập dữ liệu đã biến đổi thành một dataframe có kích thước/hình dạng giống như tập dữ liệu gốc.
# Đối với các đặc trưng đã bị loại bỏ trước đó, tập dữ liệu này sẽ bao gồm các giá trị bằng 0 thay vì giá trị gốc của chúng.
selected_features = pd.DataFrame(model.inverse_transform(X_valid_L1),
                                index=X_valid_KMeans.index,
                                columns=X_valid_KMeans.columns)

# Loại bỏ các cột từ dataframe nơi các đặc trưng được coi là không quan trọng trong việc nắm bắt phương sai của tập dữ liệu.
# Để thực hiện điều này, chúng tôi sẽ chọn lọc loại bỏ các cột có phương sai bằng 0 (tức là nơi một cột chỉ chứa các giá trị bằng 0).
X_valid_L1reg = selected_features.drop(selected_features.columns[selected_features.var() == 0], axis=1)

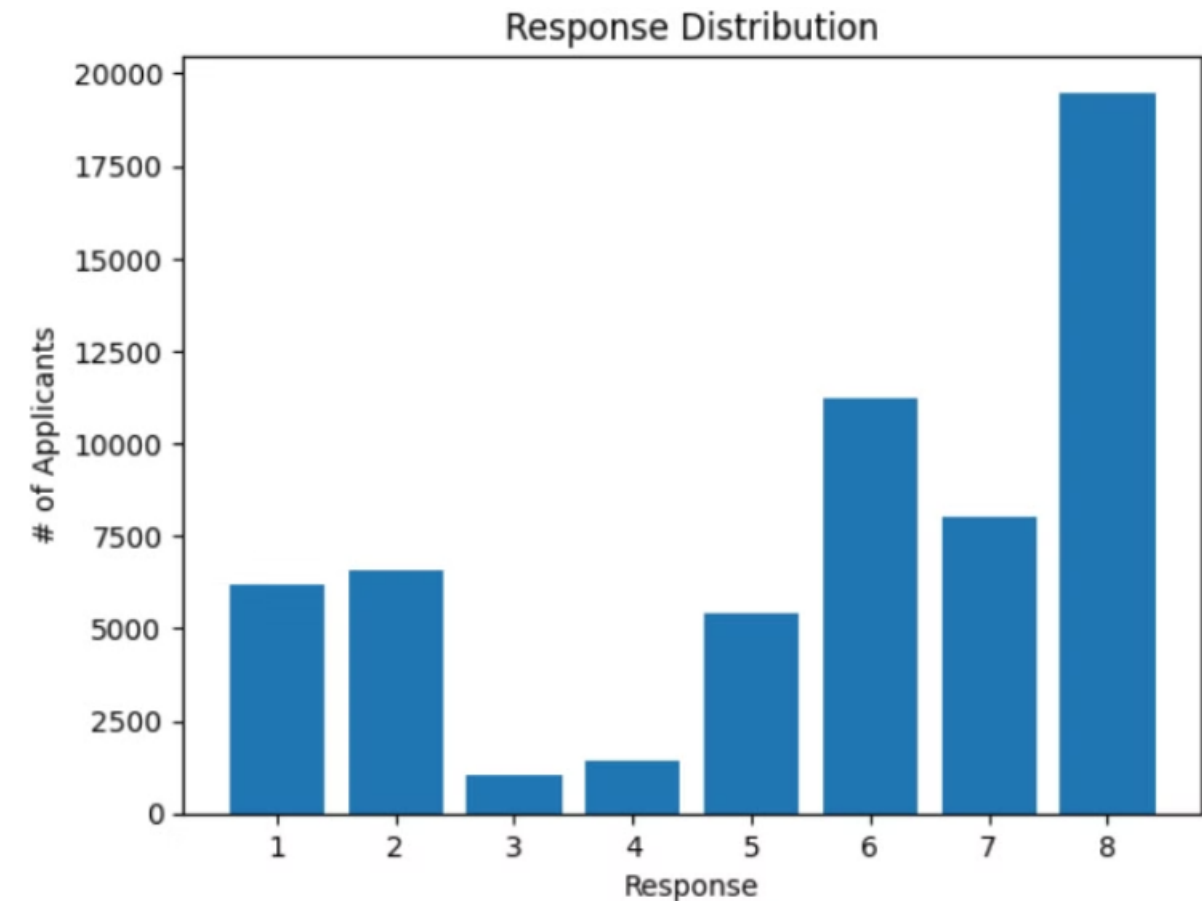
# Xác định số lượng cột được giữ lại sau khi điều chỉnh L1.
len(selected_features.columns[selected_features.var() != 0])
```

- Chọn lọc đặc trưng thông qua điều chỉnh L1 (Lasso) để xác định những đặc trưng nào trong tập dữ liệu là quan trọng nhất nhằm ngăn ngừa hiện tượng overfitting.
- Sau khi thực hiện, tập dữ liệu giảm xuống còn 57 đặc trưng từ giá trị ban đầu là 126.

Các biến và kết quả

Biến đầu vào

```
Index(['Product_Info_4', 'Product_Info_6', 'Ins_Age', 'BMI',  
      'Employment_Info_3', 'Employment_Info_5', 'InsuredInfo_4',  
      'InsuredInfo_5', 'InsuredInfo_6', 'InsuredInfo_7',  
      'Insurance_History_1', 'Insurance_History_2', 'Insurance_History_3',  
      'Insurance_History_4', 'Insurance_History_8', 'Insurance_History_9',  
      'Family_Hist_1', 'Medical_History_3', 'Medical_History_4',  
      'Medical_History_6', 'Medical_History_11', 'Medical_History_13',  
      'Medical_History_14', 'Medical_History_16', 'Medical_History_17',  
      'Medical_History_20', 'Medical_History_22', 'Medical_History_23',  
      'Medical_History_26', 'Medical_History_27', 'Medical_History_28',  
      'Medical_History_29', 'Medical_History_30', 'Medical_History_31',  
      'Medical_History_33', 'Medical_History_34', 'Medical_History_39',  
      'Medical_History_40', 'Medical_History_41', 'Medical_Keyword_3',  
      'Medical_Keyword_15', 'Medical_Keyword_23', 'Medical_Keyword_30',  
      'Medical_Keyword_37', 'Medical_Keyword_40', 'Medical_Keyword_48',  
      'Family_Hist_4', 'Product_Info_2_A6', 'Product_Info_2_A7',  
      'Product_Info_2_A8', 'Product_Info_2_D1', 'Product_Info_2_D2',  
      'Product_Info_2_D3', 'Product_Info_2_D4', 'Product_Info_2_E1',  
      'KMeansCluster_4', 'KMeansCluster_9'],  
      dtype='object')
```



Biến mục tiêu

Các biến và kết quả



Kết quả

- Response = 8 - Rủi ro **nghiêm trọng**
- Response = 7 - Rủi ro cao
- Response = 6 - Rủi ro cao
- Response = 5 - Rủi ro trung bình
- Response = 4 - Rủi ro trung bình
- Response = 3 - Rủi ro thấp
- Response = 2 - Rủi ro thấp
- Response = 1 - Rủi ro thấp

Mô hình Cây quyết định

Mô hình

Chọn mô hình Cây quyết định (Decision Tree).

Lí do

- Dễ thực thi
- Khả năng giải thích
- Bài toán phân loại đa lớp



Tối ưu hóa siêu tham số

Sử dụng phương pháp Tìm kiếm lưới toàn diện (Grid-search) để tối ưu hóa siêu tham số của mô hình. Các siêu tham số của mô hình Cây quyết định bao gồm:

- **max_depth**: đại diện cho độ sâu tối đa của cây, từ đó xác định mức độ khớp của mô hình với dữ liệu.
- **max_features**: đại diện cho số lượng đặc trưng sẽ được xem xét khi tìm kiếm phân tách tối ưu.

Các bước thực hiện:

- Khai báo lưới siêu tham số cho mô hình
- Định nghĩa một hàm tìm kiếm lưới tùy chỉnh để đo thời gian thử nghiệm
- Chạy tìm kiếm lưới tùy chỉnh và lấy các siêu tham số đã được tối ưu hóa

```
# khai báo lưới tham số cho mô hình DecisionTreeClassifier.  
param_grid_model = {'max_depth': [5, 10, 15, 20, 25, 30, 35, 40],  
                    'max_features': [0.2, 0.4, 0.6, 0.8, 1.0]}
```

```
tz = pytz.timezone('Asia/Ho_Chi_Minh')  
start = datetime.now(tz)  
print("GridSearchCV bắt đầu chạy lúc:", start.strftime("%H:%M:%S"))  
  
for i in range(0, 12):  
    GridSearchResult = GridSearcher(Model_Base, param_grid_model)  
    print(GridSearchResult.best_params_, '\n\n')  
    #GridSearchResults.append(GridSearchResult)  
  
finish = datetime.now(tz)  
print("GridSearchCV kết thúc chạy lúc:", finish.strftime("%H:%M:%S"))  
duration = finish - start  
dur = divmod(duration.seconds, 60)  
print("Tổng thời gian: ", dur[0], 'phút', dur[1], 'giây')
```

```
[CV] END .....max_depth=40, max_features=1.0; total time= 0.0s  
[CV] END .....max_depth=40, max_features=1.0; total time= 0.0s  
[CV] END .....max_depth=40, max_features=1.0; total time= 0.0s  
[CV] END .....max_depth=40, max_features=1.0; total time= 0.0s  
Kết thúc lúc: 00:07:27  
Thời gian thực hiện: 0 phút 10 giây  
{'max_depth': 10, 'max_features': 1.0}
```

Hiệu chuẩn xác suất cho mô hình

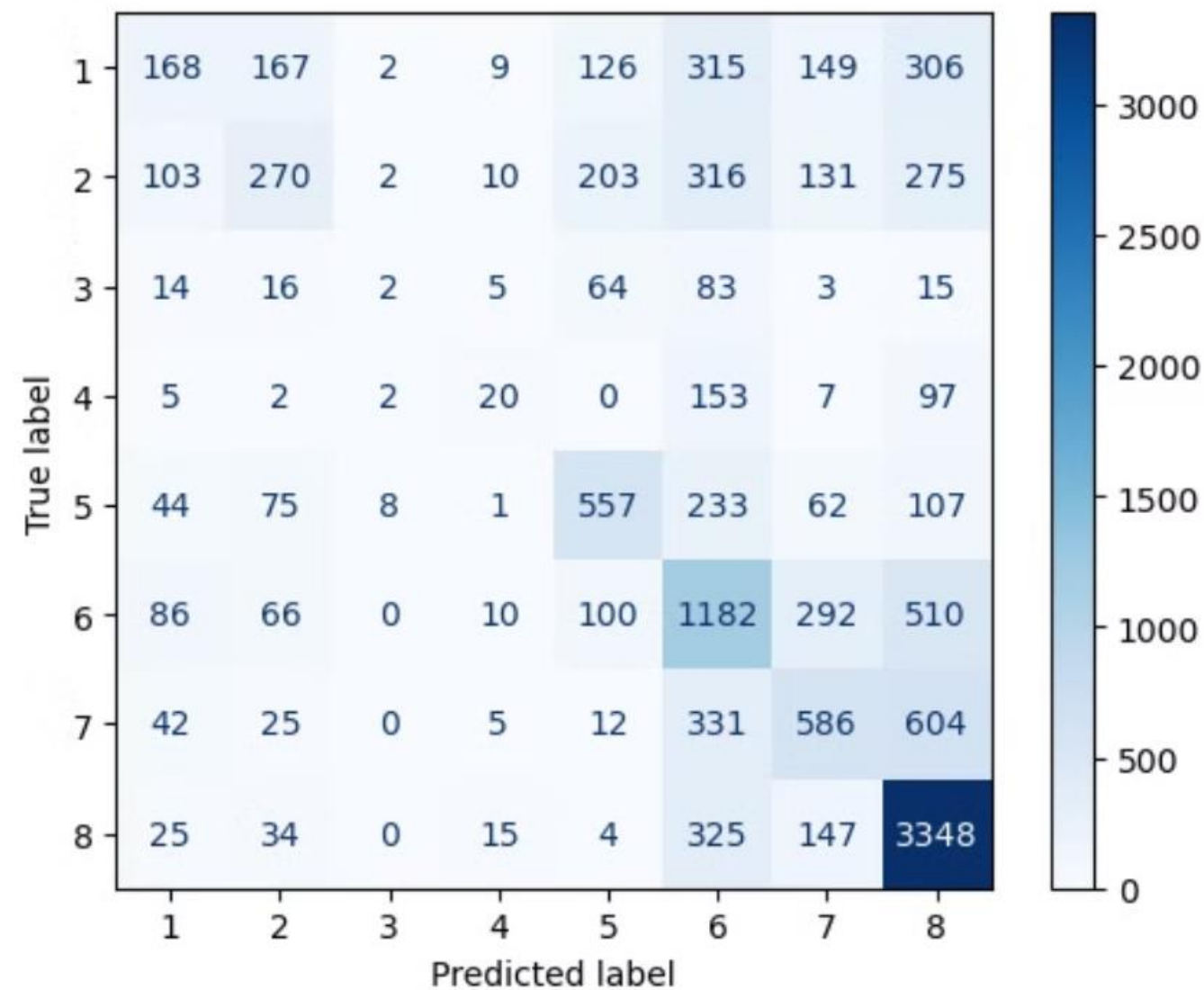
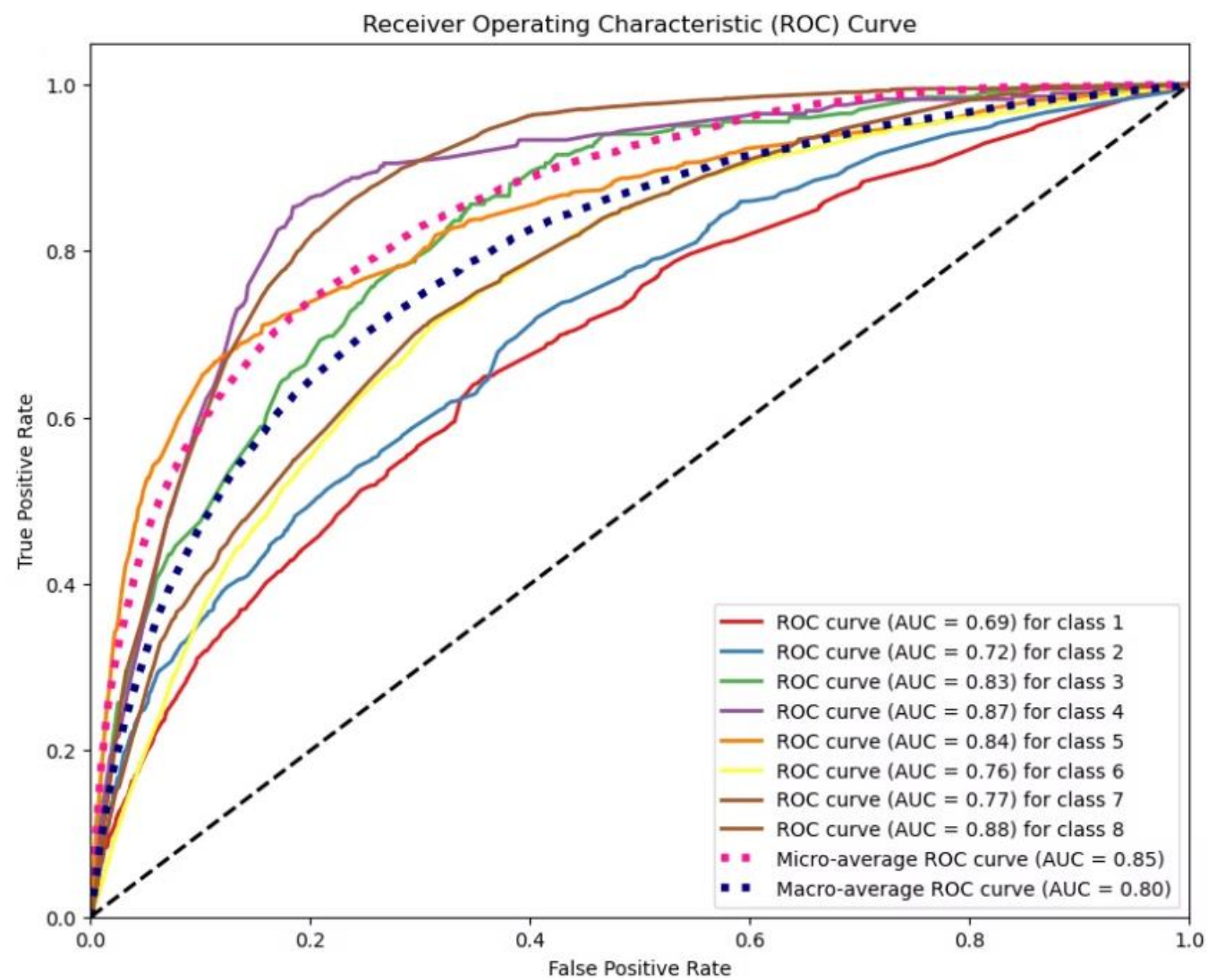
```
from sklearn.calibration import CalibratedClassifierCV
```

```
# Hiệu chuẩn bộ phân loại bằng cách sử dụng tập dữ liệu xác thực.  
Model_Calibrated = CalibratedClassifierCV(base_estimator=TrainedModel,  
                                         method="isotonic", cv="prefit").fit(X_valid_L1reg, y_valid)  
  
# method: Chỉ định phương pháp hiệu chỉnh xác suất. Ở đây, phương pháp được sử dụng là isotonic,  
# có nghĩa là một mô hình hồi quy isotonic sẽ được sử dụng để điều chỉnh xác suất.  
# cv="prefit" có nghĩa là mô hình đã được huấn luyện trước và sẽ không cần phải  
# chia dữ liệu thành các phân đoạn để thực hiện kiểm tra chéo.
```

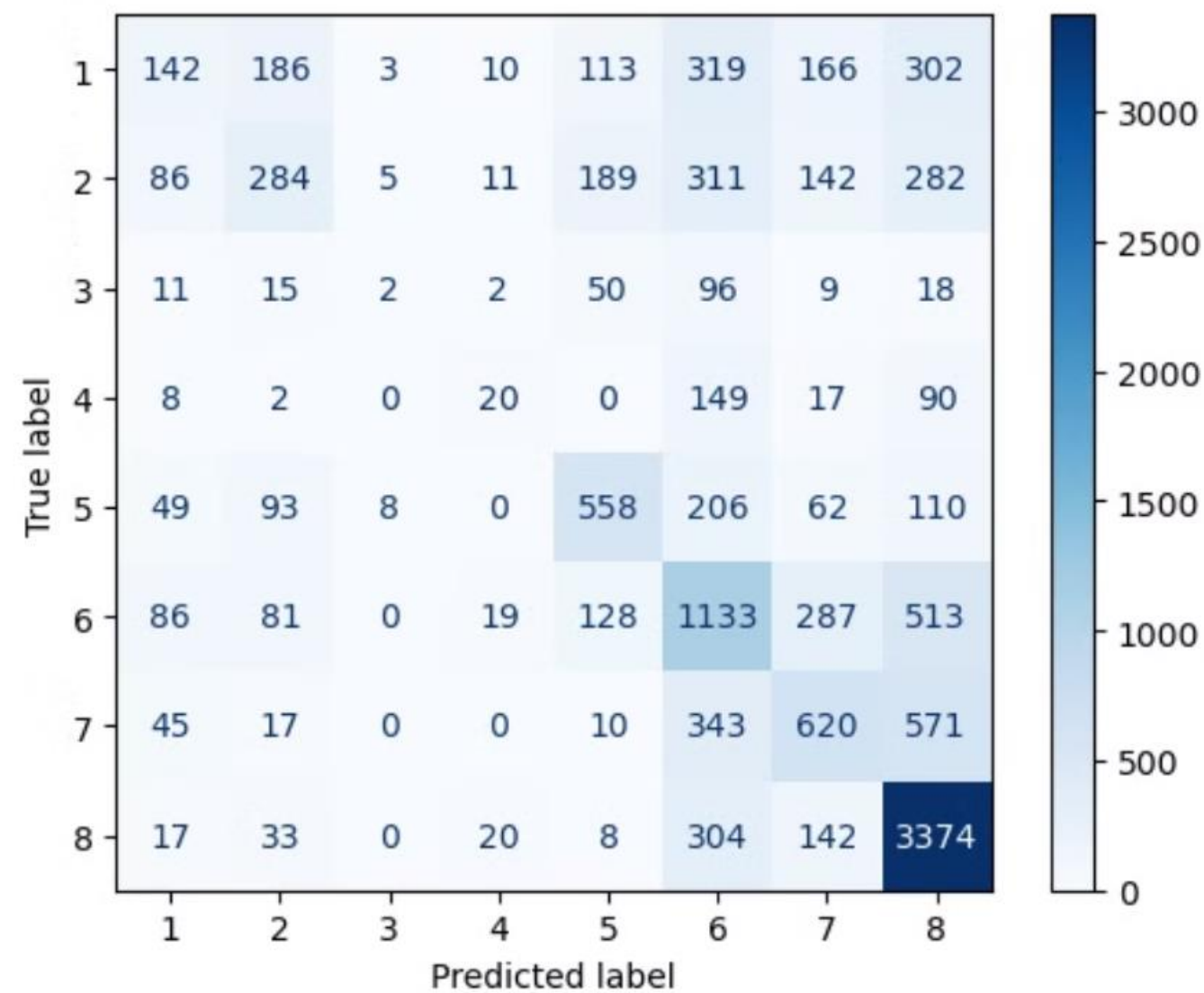
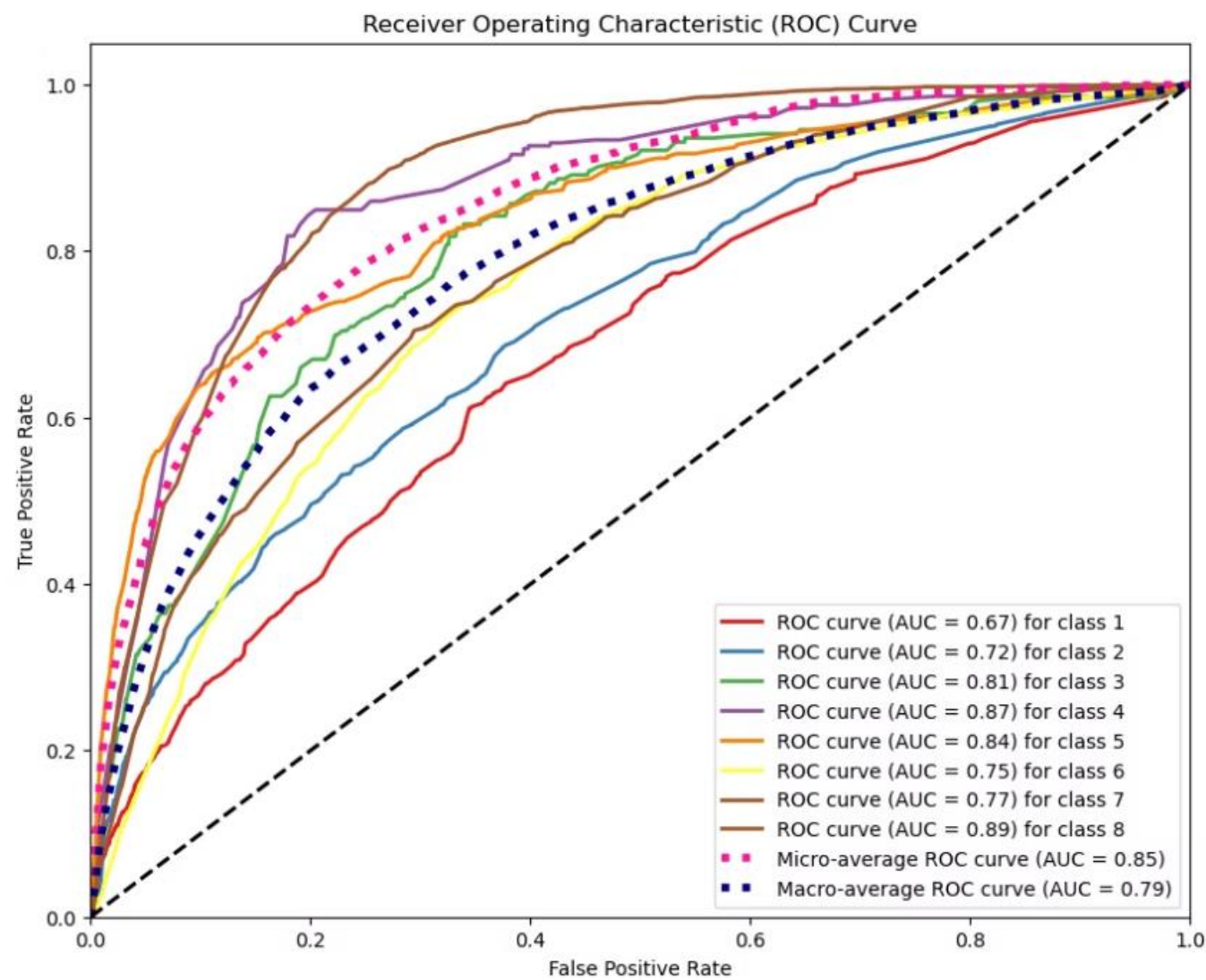
```
# Sử dụng mô hình để tạo ra các dự đoán (dưới dạng ước lượng xác suất  
# về việc thuộc về từng lớp Response) từ tập dữ liệu xác thực.  
Model_Valid_PredProb = Model_Calibrated.predict_proba(X_valid_L1reg)
```

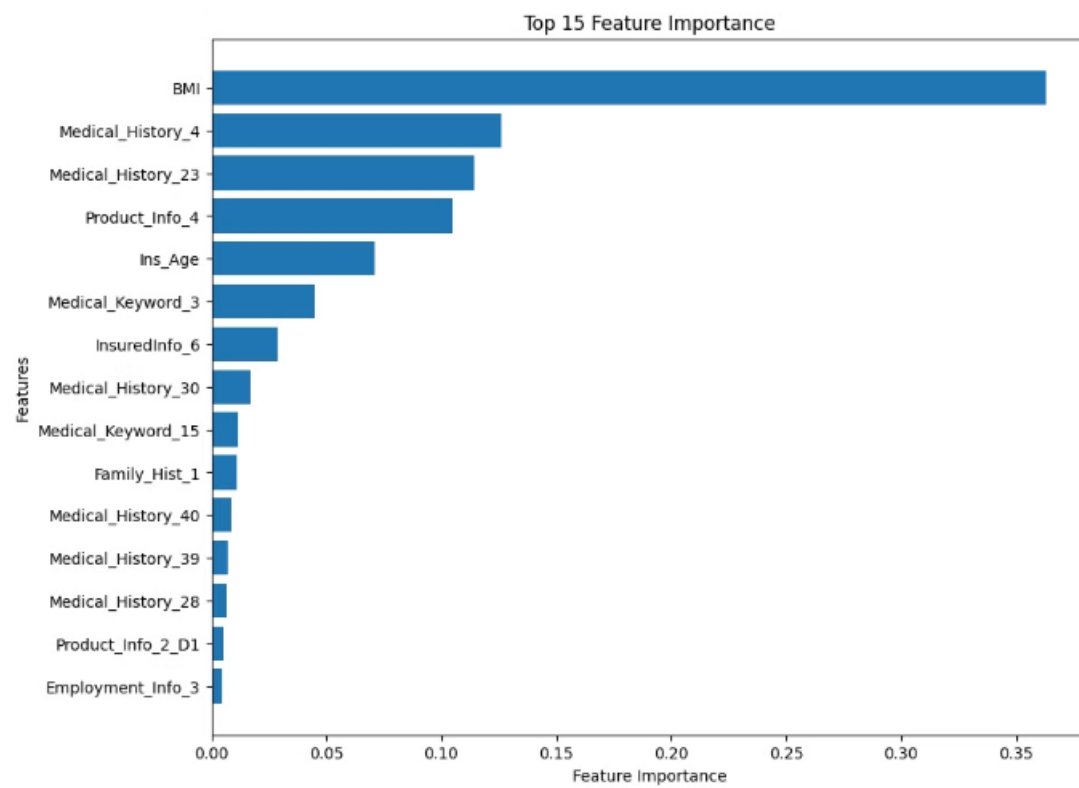
```
# Chuyển đổi các ước lượng xác suất thành dự đoán dựa trên nhãn  
  
# Model_Valid_Preds chứa các chỉ số lớp dự đoán cho mỗi mẫu trong  
# tập dữ liệu xác thực, tương ứng với xác suất cao nhất mà mô hình dự đoán.  
Model_Valid_Preds = np.argmax(Model_Valid_PredProb, axis=1)  
  
# Model_Valid_Classes chứa các nhãn mà mô hình phân loại sử dụng  
Model_Valid_Classes = Model_Calibrated.classes_  
  
# Model_Valid_Preds chứa các nhãn lớp thực tế thay vì chỉ số, giúp dễ đọc và hiểu hơn.  
Model_Valid_Preds = [Model_Valid_Classes[i] for i in Model_Valid_Preds]
```

Biểu đồ AUC và ma trận nhầm lẫn của tập Validation



Biểu đồ AUC và ma trận nhầm lẫn của tập Test





Những biến quan trọng nhất

- BMI
- Medical_History_4
- Medical_History_23
- Product_Info_4
- Ins_Age

Giá trị kinh doanh mang lại

Khi chưa áp dụng mô hình	Cụ thể
Quy Trình Thủ Công	Các nhân viên thẩm định phải xem xét và phân tích thông tin một cách thủ công, dẫn đến thời gian xử lý lâu (có thể mất hàng tuần).
Độ Chính Xác Thấp	Quyết định có thể dựa vào kinh nghiệm cá nhân mà không có sự hỗ trợ từ dữ liệu, dẫn đến nguy cơ sai lệch trong việc đánh giá rủi ro.
Chi Phí Cao	Chi phí vận hành cao do cần nhiều nhân lực và thời gian cho quy trình thẩm định.
Khả Năng Tùy Chỉnh Thấp	Khó khăn trong việc cung cấp các gói bảo hiểm tùy chỉnh phù hợp với nhu cầu từng khách hàng.

Khi chưa áp dụng mô hình

Khi áp dụng mô hình	Cụ thể
Quy Trình Tự Động Hóa	Mô hình học máy giúp tự động hóa quy trình phân loại, rút ngắn thời gian xử lý xuống còn vài ngày hoặc thậm chí vài giờ.
Độ Chính Xác Cao	Mô hình có thể phân tích và dự đoán rủi ro dựa trên dữ liệu lớn, giúp cải thiện độ chính xác trong việc đánh giá rủi ro.
Giảm Chi Phí	Giảm chi phí nhân lực và thời gian, từ đó cải thiện hiệu quả kinh doanh.
Khả Năng Tùy Chỉnh Cao	Có thể cung cấp các gói bảo hiểm tùy chỉnh dựa trên hồ sơ rủi ro cụ thể của từng khách hàng, từ đó tăng cường sự hài lòng của khách hàng.

Khi đã áp dụng mô hình